Höhere technische
Bundeslehranstalt
und Bundesfachschule
im Hermann Fuchs
Bundesschulzentrum

Ascenta - Guiding your Vision

Diplomarbeit

Schulautonomer Schwerpunkt
Coding & AI

ausgeführt im Schuljahr 2025/2026 von:

Liam Vendel, 5AHELS

Manuel Zauner, 5AHELS

Fabian Lindlbauer, 5AHELS

Betreuer:

Dipl.-Ing. Mario Preishuber

Braunau am Inn, am 11.02.2026

Eidesstattliche Erklärung

Es wird an Eides statt erklärt, dass die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als angegebene Quellen und Hilfsmittel nicht direkt benutzt und die benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht wurden.

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Liam Vendel

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Manuel Zauner

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Fabian Lindlbauer

Abstract

This diploma thesis details the development of “Ascenta”, an intelligent assistive wearable designed to support visually impaired individuals. Given the challenges of navigating complex urban environments, the project aims to create an affordable solution using artificial intelligence to provide information about the user’s social surroundings.

The technical framework utilizes the Arduino Nicla Vision microcontroller. The firmware implements the FOMO architecture for efficient on-device face detection. For user interaction, a local and privacy-oriented voice recognition system was developed using the **Vosk engine** within a custom Android application. Communication is established via an optimized UDP protocol to ensure low-latency data streaming between the wearable and the mobile device.

The current implementation successfully recognizes faces within the field of view and determines proximity using Time-of-Flight sensors. The results demonstrate the efficiency of the decentralized architecture and provide a solid foundation for future development of general obstacle detection models.

Kurzfassung

Die vorliegende Diplomarbeit dokumentiert die Entwicklung von „Ascenta“, einer intelligenten Assistenzbrille zur Unterstützung sehbeeinträchtigter Personen im Alltag. Angesichts der Komplexität moderner urbaner Umgebungen besteht das Ziel des Projekts in der Realisierung eines kostengünstigen Wearables, das mittels künstlicher Intelligenz Informationen über das soziale Umfeld liefert.

Die technische Umsetzung basiert auf dem Arduino Nicla Vision Mikrocontroller. Die Firmware nutzt die FOMO-Architektur (Faster Objects, More Objects) zur effizienten On-Device-Gesichtserkennung. Zur Interaktion wurde eine lokale, datenschutzkonforme Sprachverarbeitung auf Basis der **Vosk-Engine** in einer Android-Applikation implementiert. Die Kommunikation erfolgt über ein optimiertes UDP-Protokoll, um minimale Latenzzeiten beim Datenstreaming zwischen der Brille und dem Smartphone zu gewährleisten.

Das realisierte System ist in der Lage, Personen im Sichtfeld zu identifizieren und die Distanz mittels Time-of-Flight-Sensorik zu bestimmen. Die Ergebnisse belegen die Funktionalität der dezentralen Architektur und bilden die Grundlage für zukünftige Erweiterungen zur allgemeinen Hinderniserkennung.

Contents

Abstract iii

Kurzfassung iv

1 Einleitung 2

1.1 Motivation und Ausgangssituation 2
1.2 Problemstellung 2
1.3 Zielsetzung 2
1.4 Lösungsansatz und Kernidee 3

2 Szenarien und Use Cases 4

2.1 Use Case A: Kollisionsvermeidung mit Personen 4
2.1.1 Ausgangssituation 4
2.1.2 Systemverhalten 4
2.2 Use Case B: Kontextbezogene Umgebungsabfrage per Sprachbefehl . . 5
2.2.1 Ausgangssituation 5
2.2.2 Systemverhalten 5

3 Konzept 6

3.1 Gesamtüberblick 6
3.2 Komponenten 6
3.2.1 Mechanik und Gehäusedesign 6
3.2.2 Hardware und Elektronik 6
3.2.3 Software-Ökosystem 7
3.3 Interaktionen 7
3.4 Technisches Design 7
3.5 Evolution der Netzwerkarchitektur (Vom BLE- zum WiFi-Protokoll) . . 8
3.5.1 Initiale Konzeptionierung: Das BLE-GATT-Profil 8
3.5.2 Speichermanagement während der Audio-Aufnahme 9
3.5.3 Analyse des Flaschenhalses: MTU-Limits und OSError 11 . . . 10
3.5.4 Architektonische Neuausrichtung (TCP/IP) 11

4 Umsetzung **12**
4.1 Implementierung der Komponenten 12
4.1.1 Konstruktion und 3D-Druck 12
4.1.2 Schaltungsdesign und Energiemanagement 12
4.1.3 Entwicklung des Machine-Learning-Modells (Objekterkennung) . 12
4.1.4 Software-Architektur und Datenverarbeitung 12
4.2 Herausforderungen in der Entwicklung 23
4.2.1 Mechanische und geometrische Restriktionen 23
4.2.2 Energiemanagement im begrenzten Bauraum 23
4.2.3 Ressourcenlimits und Speicher-Fragmentierung (Software) . . . 23
4.3 Tests und Validierung 24
4.3.1 Netzwerklatenz und Streaming-Performance 24
4.3.2 Auswertung der kinetischen Interaktion (Tap-Detection) 25
4.3.3 Energetische Evaluierung (Akkulaufzeit) 25

5 Ausblick (Future Work) **27**
5.1 Software und Benutzeroberfläche 27
5.1.1 Native Zero-UI-Applikation für blinde Nutzer 27
5.1.2 Kontinuierliches auditives Raum-Feedback (Leitstrahl-Prinzip) . 27
5.2 Künstliche Intelligenz und Modell-Training 28
5.2.1 Erweiterte Objekterkennung für Alltagsgegenstände 28
5.2.2 Weiterentwicklung des Machine-Learning-Modells (Edge Impulse) 28
5.3 Hardware und Elektronik 29
5.3.1 Optimierung der Schaltungstechnik und Sensorik 29
5.4 Mechanik und Gehäusedesign 29
5.4.1 Weiterentwicklung des 3D-Modells und der Ergonomie 29

6 Verwandte Arbeiten (Related Work) **30**
6.1 Klassische Mobilitätshilfen 30
6.2 Smartphone-basierte Applikationen 30
6.3 Kommerzielle Smart Glasses und Wearables 31
6.4 Positionierung des Ascenta-Projekts 31

7 Conclusion **32**

8 Project Management **33**
8.1 Planning 33
8.2 Evaluation 33

8.3 Timesheet 34
8.3.1 Liam Vendel 34
8.3.2 Manuel Zauner 34
8.3.3 Fabian Lindlbauer 34

Acknowledgements **36**

Abbildungsverzeichnis **37**

Quellcodeverzeichnis **37**

Literaturverzeichnis **38**

CV **39**

1 Einleitung

1.1 Motivation und Ausgangssituation

Stellen Sie sich vor, Sie navigieren während der Stoßzeit durch einen belebten Bahnhof. Sie verlassen sich auf Schilder, um den Ausgang zu finden, prüfen Abfahrtstafeln und achten auf andere Pendler. Für weltweit etwa 285 Millionen sehbehinderte Menschen stellt dieses Szenario eine tägliche Herausforderung voller Barrieren dar. Während der klassische Langstock Hindernisse am Boden erkennt, versagt er bei der Identifikation von Personen oder Objekten auf Kopfhöhe. Bestehende elektronische Hilfsmittel sind oft aufgrund extrem hoher Kosten ($>$ 2.500 Euro) oder einer Abhängigkeit von permanenter Cloud-Konnektivität für die breite Masse unzugänglich.

1.2 Problemstellung

Das zentrale Problem liegt in der mangelnden digitalen Inklusion aufgrund finanzieller und technologischer Hürden. Mobile Assistenzsysteme müssen eine Echtzeit-Verarbeitung sensibler Bild- und Audiodaten ermöglichen, ohne die Ergonomie des Nutzers durch hohes Gewicht oder Wärmeentwicklung zu beeinträchtigen. Zudem erfordern Cloud-basierte Ansätze eine ständige Internetverbindung, was die Zuverlässigkeit in geschlossenen Räumen (z. B. U-Bahnhöfen) mindert und den Datenschutz gefährdet.

1.3 Zielsetzung

Das Ziel des Projekts „Ascenta“ ist die Entwicklung eines Prototyps einer smarten Assistenzbrille, die visuelle Umweltinformationen in akustisches Feedback übersetzt. Das System soll kostengünstig (Materialkosten $<$ 150 Euro) realisiert werden, eine diskrete Bauweise aufweisen und durch lokale Datenverarbeitung (Offline-Betrieb) maximale Datensicherheit für den Nutzer garantieren.

1.4 Lösungsansatz und Kernidee

Die Lösung basiert auf einer dezentralen „Split-Brain“-Architektur. Die Brille (Arduino Nicla Vision) fungiert als smarter Sensor-Satellit und führt eine ressourcensparende Objekterkennung (FOMO) zur Gesichtserkennung aus. Die rechenintensive Logik sowie die Spracherkennung werden auf das Smartphone des Nutzers ausgelagert. Die Übertragung erfolgt über ein latenzarmes WiFi-UDP-Protokoll. Durch diesen Ansatz wird die Hardware an der Brille entlastet, während die volle Prozessorleistung moderner Smartphones für die Sprachverarbeitung mittels Vosk genutzt wird.

2 Szenarien und Use Cases

Um die funktionalen Anforderungen an das Assistenzsystem „Ascenta“ zu definieren und die spätere technische Umsetzung zu rechtfertigen, werden im Folgenden konkrete Anwendungsszenarien (Use Cases) beschrieben. Diese Szenarien repräsentieren alltägliche Herausforderungen von stark sehbeeinträchtigten oder blinden Personen, die durch das System gelöst werden. Gemäß dem aktuellen Entwicklungsstand fokussieren sich die Anwendungsfälle auf die Erkennung von Personen im direkten Umfeld, die Distanzabschätzung sowie die auditive Interaktion.

2.1 Use Case A: Kollisionsvermeidung mit Personen

2.1.1 Ausgangssituation

Ein Nutzer bewegt sich mit einem regulären Langstock durch einen öffentlichen Raum, beispielsweise ein Foyer oder einen Gehweg. Während der Langstock Bodenhindernisse (wie Treppenstufen oder Bordsteinkanten) zuverlässig ertastet, registriert er keine stillstehenden oder sich nähernden Personen auf Körper- oder Kopfhöhe, bevor es zu einem physischen Kontakt kommt.

2.1.2 Systemverhalten

Das Kamerasystem der Ascenta-Brille erfasst kontinuierlich das Sichtfeld des Nutzers. Sobald eine Person in den Erfassungsbereich tritt, wird diese durch das lokale Machine-Learning-Modell (FOMO) auf dem Mikrocontroller identifiziert. Simultan misst der Time-of-Flight-Sensor (ToF) die exakte Distanz zu diesem Objekt.

Unterschreitet die Person eine vordefinierte kritische Distanz (beispielsweise einen Meter), fusioniert die Smartphone-Applikation diese Sensordaten und generiert unmittelbar ein akustisches Warnsignal. Der Nutzer wird dadurch in Echtzeit auf das Hindernis aufmerksam gemacht und kann rechtzeitig ausweichen, ohne dass ein physischer Kontakt entsteht.

2.2 Use Case B: Kontextbezogene Umgebungsabfrage per Sprachbefehl

2.2.1 Ausgangssituation

Der Nutzer befindet sich in einer Warteschlange oder an einem Haltepunkt (z. B. einer Bushaltestelle) und möchte wissen, ob sich eine Person direkt vor ihm befindet, um entsprechend aufrücken oder Abstand halten zu können. Ein weites Ausschwenken des Langstocks ist in dieser Situation unerwünscht, da es andere Personen stören oder irritieren könnte.

2.2.2 Systemverhalten

Anstatt die Umgebung physisch abzutasten, nutzt der Anwender die integrierte Sprachsteuerung. Der Nutzer aktiviert das System (beispielsweise durch ein definiertes Interaktionsmuster an der Brille) und stellt verbal eine Frage, wie etwa: „Befindet sich jemand vor mir?“.

Das Audiosignal wird über das in der Brille verbaute Mikrofon erfasst und via UDP-Protokoll latenzarm an das verbundene Smartphone gestreamt. Dort verarbeitet die lokale Vosk-Engine den Sprachbefehl offline, um den Datenschutz zu gewährleisten. Die Applikation gleicht die Anfrage mit den aktuellen Kameradaten (Personenerkennung) und der Distanzmessung (ToF) ab. Anschließend erhält der Nutzer über die Sprachausgabe (Text-to-Speech) ein diskretes, akustisches Feedback, beispielsweise: „Person in einem Punkt fünf Metern Entfernung erkannt.“ Dadurch gewinnt der Nutzer ohne stigmatisierende Bewegungen ein präzises Verständnis seiner unmittelbaren sozialen Umgebung.

3 Konzept

Dieses Kapitel beschreibt die abstrakte Architektur des Assistenzsystems „Ascenta“. Es liefert einen Gesamtüberblick über das funktionale Zusammenwirken der Teilsysteme und erläutert die iterativen Designentscheidungen, welche zur finalen Netzwerktopologie führten.

3.1 Gesamtüberblick

Das System basiert auf einer dezentralen „Split-Brain“-Architektur. Um die Hardware direkt am Kopf des Nutzers so leicht und unauffällig wie möglich zu gestalten, lagert die Architektur die ressourcenintensive Datenverarbeitung auf das Smartphone des Nutzers aus. Die Brille fungiert als reiner Sensor- und Kommunikationsknotenpunkt (Edge-Device). Sie erfasst Umgebungsdaten und streamt die Ergebnisse einer lokalen Vorverarbeitung an das mobile Endgerät. Das Smartphone übernimmt die Rolle des Hauptrechenzentrums. Es fusioniert die Sensordaten, interpretiert Benutzereingaben und generiert das akustische Feedback.

3.2 Komponenten

Das Gesamtsystem setzt sich aus drei wesentlichen logischen Bausteinen zusammen: der physischen Struktur (Gehäuse), der elektronischen Hardware (Sensorik und Energieversorgung) und dem Software-Ökosystem.

3.2.1 Mechanik und Gehäusedesign

[Hier fügt Manuel seine konzeptionellen Gedanken zum Gehäusedesign und den ergonomischen Anforderungen ein (Warum 3D-Druck, Gewichtsverteilung, etc.).]

3.2.2 Hardware und Elektronik

[Hier fügt Fabian die konzeptionelle Begründung der Elektronik und Bauteilwahl ein (Warum Nicla Vision, warum LiPo-Akkus, grobes Energiekonzept).]

3.2.3 Software-Ökosystem

Die Softwarearchitektur teilt sich in die Firmware des Mikrocontrollers und die mobile Applikation. Die Firmware (MicroPython) verantwortet die hardwarenahe Sensorsteuerung und eine erste Datenreduktion (Edge AI mittels FOMO). Die Android-Applikation (Kotlin) dient als unsichtbares Interface (Zero-UI) und integriert komplexe Algorithmen wie die Offline-Spracherkennung (Vosk) zur auditiven Interaktion.

3.3 Interaktionen

Die Interaktion zwischen den Systemkomponenten und dem Nutzer verläuft in einem kontinuierlichen Kreislauf:

1. **Datenerfassung:** Kamera und Time-of-Flight-Sensor erfassen synchron die visuelle Umgebung und Distanzdaten. Parallel überwacht die Inertial Measurement Unit (IMU) physische Eingaben durch den Nutzer.
2. **Vorverarbeitung und Übertragung:** Der Mikrocontroller identifiziert relevante Objektkoordinaten. Er überträgt diese asynchron über ein lokales WiFi-Netzwerk an das Smartphone und ergänzt den Stream bei Bedarf durch Audio-Fragmente.
3. **Fusion und Auswertung:** Das Smartphone verknüpft die visuellen Koordinaten mit den Distanzwerten und wertet eventuelle Sprachbefehle aus.
4. **Feedback:** Überschreitet ein Objekt einen kritischen Schwellenwert oder erfordert ein Sprachbefehl eine Antwort, generiert das Smartphone ein auditives Signal oder eine Sprachausgabe (TTS). Der Nutzer nimmt dieses Feedback unmittelbar über die Lautsprecher oder Kopfhörer wahr.

3.4 Technisches Design

Das technische Design der Kommunikation stellt sicher, dass kritische Warnungen höchste Priorität erhalten. Die Architektur nutzt das User Datagram Protocol (UDP) bewusst für Discovery-Prozesse und das verbindungsorientierte TCP für den asynchronen Binärdaten-Stream. Zudem garantiert die konsequente Offline-Ausführung aller Machine-Learning-Modelle den Schutz sensibler Nutzerdaten (Privacy-by-Design). Das

System übermittelt zu keinem Zeitpunkt Umgebungsvideos oder Audiodaten an externe Cloud-Anbieter zur Verarbeitung.

3.5 Evolution der Netzwerkarchitektur (Vom BLE- zum WiFi-Protokoll)

Die finale Netzwerktopologie (UDP/TCP) resultiert aus einer iterativen Evaluierung und umfassenden Performancetests verschiedener Übertragungsstandards. Frühe Prototypen des Systems nutzten ein vollumfängliches Bluetooth Low Energy (BLE) Setup für den gesamten Datentransfer (Telemetrie, Bilder, Audio). Diese Architektur erwies sich in der Praxis jedoch als massiver Flaschenhals und erzwang einen Paradigmenwechsel. Die folgenden Abschnitte dokumentieren diese architektonische Evolution.

3.5.1 Initiale Konzeptionierung: Das BLE-GATT-Profil

Das ursprüngliche Konzept sah das Nicla Vision als eigenständiges BLE-Peripheral vor. Das System spannte einen dedizierten GATT-Server (Generic Attribute Profile) auf und definierte spezifische Services für die Kommunikation mit dem Smartphone.

Die Architektur gliederte den Datenaustausch in drei logische *Services* mit jeweils eigenen 128-Bit-UUIDs (Universally Unique Identifiers): Einen Service für Bilddaten (`NICLA_SERVICE`), einen für Steuerbefehle (`CMD_SERVICE`) und eine UART-Emulation (`UART_SERVICE`) für den bidirektionalen Transfer von Audiodaten. Das System verknüpfte diese Services mit *Characteristics*, welche die Lese- und Schreibrechte (Notify, Read, Write) auf Hardware-Ebene definierten:

```
import bluetooth

# Definition der Service- und Characteristic-UUIDs fuer die UART-Emulation
UART_SERVICE_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
UART_TX_UUID      = bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E")
UART_RX_UUID      = bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E")

```

```
# Registrierung der Services im BLE-Stack des Mikrocontrollers
uart_svc = (UART_SERVICE_UUID, (
    (UART_TX_UUID, bluetooth.FLAG_NOTIFY | bluetooth.FLAG_READ),
    (UART_RX_UUID, bluetooth.FLAG_WRITE),
))
handles = self.ble.gatts_register_services((vision_svc, cmd_svc,
    uart_svc))
```

Listing 3.1: Auszug der initialen GATT-Service-Definitionen für die alte BLE-Architektur

3.5.2 Speichermanagement während der Audio-Aufnahme

Ein spezifisches Problem der BLE-Datenübertragung war die Notwendigkeit, Audiodaten vor dem Senden vollständig im Arbeitsspeicher (RAM) zwischenzuspeichern. Da das BLE-Protokoll asynchron arbeitet und keine konstante Datenrate garantiert, scheiterte ein direktes Live-Streaming des Mikrofons.

Die Firmware implementierte daher eine speicherintensive *Record-to-RAM*-Logik. Das Skript berechnete den benötigten Speicherplatz anhand der Samplingrate (16.000 Hz), der Auflösung (16-Bit) und der Kanäle (Mono). Eine viersekündige Aufnahme beanspruchte somit exakt 128.000 Bytes. Um *Out-Of-Memory*-Abstürze zu verhindern, fragmentierte der Code den RAM in Arrays zu je 4096 Bytes (`CHUNK_SIZE`) und deaktivierte den internen Garbage-Collector (`gc.disable()`) während der Aufnahme, um Audio-Aussetzer durch blockierende Interrupts zu vermeiden:

```
# Berechnung des Gesamtspeichers: 16000 Hz * 1 Channel * 2 Bytes * 4.0s
TOTAL_BYTES = int(SAMPLE_RATE * CHANNELS * BYTES_PER_SAMPLE *
    RECORD_SECONDS)
CHUNK_SIZE = 4096
NUM_CHUNKS = (TOTAL_BYTES // CHUNK_SIZE) + 1

def record_to_ram():
    global g_chunks, g_is_recording
    gc.collect()
    g_chunks = []

    # Fragmentierte Allozierung zur Verhinderung von OOM
    for i in range(NUM_CHUNKS):
        g_chunks.append(bytearray(CHUNK_SIZE))

    g_is_recording = True
```

```
    gc.disable() # Kritisch: GC deaktivieren, um Interrupt-Blockaden zu
        meiden
    audio.start_streaming(audio_callback)

    while g_is_recording:
        time.sleep_ms(50)

    audio.stop_streaming()
    gc.enable()
```

Listing 3.2: RAM-Allokation und Pufferung der Audiodaten in der initialen Firmware

3.5.3 Analyse des Flaschenhalses: MTU-Limits und OSError 11

Die Übertragung der generierten 128 KB über den BLE-Stack offenbarte kritische Limitierungen. Das Bluetooth Low Energy Protokoll beschränkt die Payload-Größe pro Datenpaket (Maximum Transmission Unit, MTU) enorm. Selbst nach einer erfolgreichen MTU-Aushandlung mit dem Android-Smartphone (`MTU Exchange`) betrug die maximale Payload-Größe pro Paket meist unter 250 Bytes. Im Worst-Case-Szenario sank dieser Wert auf den BLE-Standard von 20 Bytes (`self.payload_size = 20`).

Das System zerlegte die Binärdaten mithilfe von `memoryview` in mikroskopische Chunks und schickte diese über die Methode `gatts_notify` sequenziell an das mobile Endgerät. Dies führte unweigerlich zu einer Überlastung des internen Sende-Puffers auf dem Hardware-Chip des Nicla Vision, was der MicroPython-Interpreter mit einem `OSError 11` (EAGAIN / Buffer Full) quittierte. Um diesen Absturz zu verhindern, erzwang die Sende-Schleife einen „künstlichen“ Delay von 15 Millisekunden nach jedem Sendevorgang:

```
def send(self, data):
    mv = memoryview(data)
    bytes_sent = 0
    total_len = len(data)

    while bytes_sent < total_len:
        if not self.conn: break
        # Limitierung auf die ausgehandelte MTU-Payload (z.B. 20 - 240
            Bytes)
        chunk_len = min(self.payload_size, total_len - bytes_sent)

        try:
```

```
            self.ble.gatts_notify(self.conn, self.h_tx, mv[bytes_sent :
                bytes_sent + chunk_len])
            bytes_sent += chunk_len
        except OSError:
            pass # Puffer voll: Ausnahme ignorieren und im naechsten
                Zyklus iterieren

        # Zwangspause von 15ms zur Verhinderung eines Hardware-Overflows
        time.sleep_ms(15)
```

Listing 3.3: Die BLE-Sendeschleife mit künstlicher Drosselung zur Puffer-Schonung

Diese mathematische Kombination aus winzigen Datenpaketen und künstlichen Verzögerungen machte ein Echtzeitsystem unmöglich. Geht man von einer Payload von 240 Bytes aus, zersplittert die 128 KB große Audiodatei in über 530 Einzelpakete. Allein die summierten Zwangspausen (530×15 ms) verursachen einen Sende-Overhead von knapp acht Sekunden – zuzüglich der eigentlichen Funkübertragungsdauer. Im Fall einer 20-Byte-MTU explodiert die Übertragungszeit auf weit über eine Minute. Für ein Assistenzsystem im Straßenverkehr, welches Blinde verzögerungsfrei vor Hindernissen warnen muss, verfehlt eine derart hohe Latenz die fundamentalen Sicherheitsanforderungen.

3.5.4 Architektonische Neuausrichtung (TCP/IP)

Die evaluierten Messwerte und Puffer-Überläufe erzwangen einen grundlegenden Paradigmenwechsel in der Systemarchitektur. Das aktuelle System limitiert die Nutzung von Bluetooth Low Energy strikt auf das initiale Verbindungs-Setup, da BLE für kleine Text-Credentials (SSID und Passwort) hervorragend geeignet ist.

Den eigentlichen Hauptdatenstrom (Audio-Chunks und JPEG-Bilder) verlagert die Brille jedoch auf ein lokales WiFi-Netzwerk unter Nutzung des Transmission Control Protocols (TCP). Dieser Wechsel erhöht die Bandbreite exponentiell, beseitigt die künstlichen Sende-Delays und drückt die Systemlatenz für komplexe Binärübertragungen von mehreren Sekunden auf den benötigten Bruchteil einer Sekunde.

4 Umsetzung

Das vorherige Kapitel definiert das theoretische Konzept. Dieses Kapitel beschreibt die praktische Realisierung des Prototyps.

4.1 Implementierung der Komponenten

4.1.1 Konstruktion und 3D-Druck

[Hier fügt Manuel seinen sachlichen Text zur CAD-Modellierung in Fusion 360, den Maßen, dem Klappmechanismus und dem Druckvorgang ein.]

4.1.2 Schaltungsdesign und Energiemanagement

[Hier fügt Fabian seine Schaltpläne, die genaue Verkabelung (TP4056, Nicla) und die Entladungskurven der Akkus ein.]

4.1.3 Entwicklung des Machine-Learning-Modells (Objekterkennung)

[Hier fügt Fabian seinen Text zum Training des Modells mit Roboflow, Kaggle und Edge Impulse ein.]

4.1.4 Software-Architektur und Datenverarbeitung

Die softwareseitige Umsetzung erfordert die Koordination eines ressourcenbeschränkten Embedded-Systems (MicroPython) und eines mobilen Betriebssystems (Android). Die folgenden Abschnitte erläutern die Lösungsansätze für die Netzwerkkommunikation, die asynchrone Datenverarbeitung und das Speichermanagement detailliert.

Android Lifecycle-Management und Systementkopplung

Da das Smartphone während der Nutzung der Assistenzbrille typischerweise in der Tasche des Nutzers verbleibt und der Bildschirm deaktiviert ist, neigt das Android-

Betriebssystem dazu, Hintergrundprozesse aus Energiespargründen zu terminieren (Doze-Mode). Die Architektur lagert daher die gesamte Netzwerk- und Bluetooth-Logik in einen Android `Service` (`AscentaService.kt`) aus.

Der Aufruf von `startForeground()` signalisiert dem System höchste Priorität. Ein weiteres kritisches Problem stellt der Standby-Modus dar, der den Empfang von UDP-Broadcast-Paketen auf Hardware-Ebene blockiert. Die Applikation löst dies durch die Akquirierung eines `WifiManager.MulticastLock`. Dieser Lock zwingt den WiFi-Chip, Broadcast-Pakete auch bei deaktiviertem Display durchzureichen:

```
val wifiManager = applicationContext.getSystemService(Context.
    WIFI_SERVICE) as WifiManager
multicastLock = wifiManager.createMulticastLock("AscentaUDP")
multicastLock?.setReferenceCounted(true)
multicastLock?.acquire()

startForeground(1, createNotification("Ascenta Active"))
```

Listing 4.1: Akquirierung des MulticastLocks und Start des Foreground Services

Asynchrone BLE-GATT State-Machine

Das System überträgt die WLAN-Credentials (Provisioning) initial über das Bluetooth Low Energy (BLE) Generic Attribute Profile (GATT) vom Smartphone auf die Brille. Da Bluetooth-Operationen in Android stark asynchron und Event-getrieben ablaufen, implementiert der Code eine komplexe State-Machine innerhalb eines `BluetoothGattCallback`.

Sobald der `AscentaService` die Brille über den BLE-Scanner findet, baut er eine GATT-Verbindung auf. Das Betriebssystem meldet Statusänderungen an die Methode `onConnectionStateChange`. Bei erfolgreicher Verbindung initiiert die Applikation die *Service Discovery*. Erst wenn die Services des Nicla Vision im Android-System registriert sind, schreibt die Applikation die SSID als Bytearray in die spezifische Characteristic. Ein erfolgreicher Schreibvorgang triggert `onCharacteristicWrite`. Daraufhin überträgt das System kaskadierend das Passwort und terminiert die BLE-Verbindung sauber:

```
override fun onServicesDiscovered(g: BluetoothGatt, s: Int) {
    val svc = g.getService(PROV_SERVICE_UUID)
    val ssidChar = svc?.getCharacteristic(SSID_CHAR_UUID)
    if (ssidChar != null) {
        ssidChar.value = ssid.toByteArray()
```

```
        g.writeCharacteristic(ssidChar) // Triggert asynchronen Schreibvorgang
    }
}

override fun onCharacteristicWrite(g: BluetoothGatt, char: BluetoothGattCharacteristic, status: Int) {
    if (status == BluetoothGatt.GATT_SUCCESS && char.uuid == SSID_CHAR_UUID) {
        val passChar = g.getService(PROV_SERVICE_UUID).getCharacteristic(PASS_CHAR_UUID)
        Thread.sleep(100) // Hardware-Pufferung des Nicla Vision abwarten
        passChar.value = pendingPass!!.toByteArray()
        g.writeCharacteristic(passChar)
    }
}
```

Listing 4.2: Ausschnitt der asynchronen GATT Callback-Struktur in Kotlin

Hybrides Netzwerkprotokoll (UDP & TCP)

Nach erfolgreichem BLE-Provisioning verbindet sich der Mikrocontroller mit dem WLAN und sendet periodisch `NICLA_READY`-Broadcasts via UDP auf Port 5006. Eine Kotlin-Coroutine in der App lauscht asynchron auf diese Pakete und ermittelt die dynamisch vergebene IP-Adresse der Brille.

```
udpSocket = DatagramSocket(UDP_DISC_PORT).apply { broadcast = true }
val packet = DatagramPacket(buffer, buffer.size)

while (isActive && !isConnected) {
    udpSocket?.receive(packet)
    val str = String(packet.data, 0, packet.length).trim()
    if (str == "NICLA_READY") {
        udpSocket?.close()
        connectTcp(packet.address) // TCP-Handshake einleiten
        break
    }
}
```

Listing 4.3: Asynchroner UDP-Discovery-Listener in Kotlin

Nach der IP-Ermittlung etabliert die App den Hauptdatenstrom über eine verbindungsorientierte TCP-Verbindung (Port 5005). Die Architektur nutzt hierbei TCP anstelle

von UDP, da dieses Protokoll Paketverluste bei der Übertragung großer binärer Payloads (WAV-Audio-Chunks oder JPEG-Bilder) strikt verhindert. Die Firmware konfiguriert den Socket auf dem Nicla Vision mittels `setblocking(False)`. Dies verhindert, dass abgebrochene Verbindungen den Mikrocontroller in einen Deadlock versetzen.

Stream-Multiplexing und Custom Protocol Parser

Die Nutzung einer einzigen TCP-Verbindung (Port 5005) für völlig unterschiedliche Datentypen kennzeichnet eine architektonische Besonderheit des Systems. Der Datenstrom (*Stream*) interleavt kontinuierliche Textnachrichten (JSON-Telemetrie, Heartbeat-Pongs) mit großen binären Payloads (JPEG-Bilder, WAV-Audiodaten).

Ein eigenes, leichtgewichtiges Applikationsprotokoll (Application Layer) verhindert eine Vermischung der Datentypen. Der Code stellt binären Übertragungen einen definierten Präfix inklusive Payload-Größe voran (z. B. `IMG_START:12500`). Der Android `AscentaService` implementiert einen maßgeschneiderten Parser. Dieser Parser liest den eingehenden `InputStream` zunächst zeichenweise (Byte für Byte) bis zum Line-Feed (`\n`) aus. Erkennt der Parser einen Binär-Präfix, schaltet er den Lesemodus dynamisch um, extrahiert exakt die angegebene Anzahl an Bytes in ein `ByteArray` und wechselt danach wieder in den Text-Modus:

```
// Auslesen des TCP-InputStreams in der Coroutine
val line = readLineFromStream(input) // Zeichenweises Lesen bis \n

if (line.startsWith("IMG_START:")) {
    val size = line.substringAfter(":").toInt()
    val bytes = readBytesFromStream(input, size) // Blockweises Lesen
    val bmp = BitmapFactory.decodeByteArray(bytes, 0, bytes.size)
    if (bmp != null) handler.post { callback?.onImageReceived(bmp) }

} else if (line.startsWith("AUD_START:")) {
    val size = line.substringAfter(":").toInt()
    val bytes = readBytesFromStream(input, size)
    saveAudioForVosk(bytes)

} else if (line.startsWith("{")) {
    // Standard JSON-Parsing fuer Sensordaten
}
```

Listing 4.4: Dynamisches Stream-Multiplexing zur Trennung von Text- und Binärdaten

Ausfallsicherheit durch Heartbeat-Mechanismus

Die `MainActivity` implementiert einen asynchronen Watchdog, um Verbindungsabbrüche zuverlässig zu erkennen. Die Applikation sendet alle zehn Sekunden einen Ping an die Brille. Bleibt der Response für mehr als 25 Sekunden aus, schließt die Software die Verbindung sicher und reaktiviert den UDP-Discovery-Modus (*Self-Healing*):

```
heartbeatJob = CoroutineScope(Dispatchers.IO).launch {
    while (isActive) {
        if (isConnected) {
            sendCommand("ping")
            delay(10000)
            if (System.currentTimeMillis() - lastPingResponse > 25000) {
                withContext(Dispatchers.Main) {
                    isConnected = false
                    stopHeartbeat()
                    showHome() // Zurueck in den Discovery-Status
                }
            }
        } else { delay(5000) }
    }
}
```

Listing 4.5: Heartbeat-Coroutine zur Überwachung der Verbindungsstabilität

Concurrency und Thread-Sicherheit in Kotlin

Die Android-Applikation führt netzwerkbasierte I/O-Operationen (TCP/UDP-Sockets) und rechenintensive Machine-Learning-Prozesse (Vosk) simultan aus. Dafür implementiert der Code ein striktes Thread-Dispatching mittels *Coroutines*. Sämtliche Sockets und blockierende Lesevorgänge operieren ausschließlich im `Dispatchers.IO` Thread-Pool. Sobald die Coroutine ein Datenpaket vollständig empfängt, führt sie einen synchronisierten Kontextwechsel zurück auf den Main-Thread durch (`handler.post`). Der Main-Thread aktualisiert anschließend die Benutzeroberfläche oder steuert die TTS-Engine an:

```
scope.launch(Dispatchers.IO) { // Start im Hintergrund-Thread
    val input = tcpSocket!!.getInputStream()
    while (isActive && isConnected) {
        val line = readLineFromStream(input) // Blockierender I/O-Aufruf

        if (line.startsWith("{")) {
            val json = JSONObject(line)
```

```
            val label = json.optString("label")

            // Sicherer Wechsel auf den Main-Thread fuer das UI/Audio-
                Feedback
            handler.post {
                callback?.onDetectionResult(label, conf, dist)
            }
        }
    }
}
```

Listing 4.6: Sicherer Kontextwechsel zwischen IO-Thread und Main-Thread in Kotlin

Ressourcenmanagement und Modell-Integration (Nicla Vision)

Die Firmware des Nicla Vision (`Firmware.py`) operiert unter starken Speicherrestriktionen. Das System reserviert initial einen Notfall-Puffer (`micropython.alloc_emergenc` um Out-of-Memory (OOM) Exceptions während Laufzeit-Interrupts zu verhindern. Das Skript integriert das trainierte FOMO-Modell ressourceneffizient direkt im Framebuffer des Mikrocontrollers (`load_to_fb=True`).

Die Kernschleife der Umgebungserfassung

Die Hauptmethode `run_active_detection` auf dem Mikrocontroller steuert die kontinuierliche Überwachung der Umgebung und die Peripherie. Diese Funktion minimiert Latenzen und vermeidet blockierende Aufrufe.

In jedem Iterationsschritt erfasst der Sensor ein neues Kamerabild (`sensor.snapshot()`) und übergibt dieses an das geladene Machine-Learning-Modell (`net.predict()`). Parallel wertet das System die Distanz über den I2C-Bus des VL53L1X aus. Erkennt der Algorithmus ein valides Objekt mit einer Konfidenz über dem Schwellenwert, konstruiert die Firmware dynamisch einen JSON-String. Dieser String bündelt das Label, den Konfidenzwert sowie die gemessene Distanz. Anschließend streamt das System dieses Paket asynchron über den TCP-Socket:

```
img = sensor.snapshot()
distance_mm = 0
if self.tof:
    try: distance_mm = self.tof.read()
    except: pass

```

```
raw_output = self.net.predict([img])
# ... (Parsing der Heatmap, Bestimmung von best_label und best_conf) ...

if best_label:
    json_str = '{"label": "%s", "conf": "%.2f", "dist": "%d", "x": "%d",
        "y": "%d"}\n' % (
        best_label, best_conf, distance_mm, best_x, best_y
    )
    self.send_tcp_packet(json_str.encode())

gc.collect() # Proaktive Defragmentierung des Speichers
```

Listing 4.7: Extraktion der Kameradaten und asynchroner JSON-Telemetrie-Stream

Auswertung des 4D-Tensors und Sensorfusion

Die Auswertung der Vorhersagedaten des FOMO-Modells stellt eine besondere algorithmische Herausforderung dar. Das quantisierte Modell generiert einen vierdimensionalen Tensor (Heatmap), der das Bild in ein Grid unterteilt. Die Firmware iteriert über das X/Y-Grid. Findet der Algorithmus ein lokales Maximum über dem Schwellenwert $(0,45)$, berechnet die Firmware den absoluten Bildschwerpunkt (Zentroid) und fusioniert die 2D-Bildkoordinaten mit der Z-Achse (Distanz in Millimetern) des ToF-Sensors.

Low-Level Hardware-Kommunikation (I2C & Registerauslesung)

Die Firmware liest den integrierten Fuel-Gauge-IC (MAX17262) über den I2C-Bus (Inter-Integrated Circuit) aus, um dem Nutzer zuverlässige Batteriewarnungen zu geben. Der Mikrocontroller adressiert den Chip unter der Hexadezimal-Adresse `0x36` und fordert zwei Bytes ab dem Register `0x06` (RepSOC) an. Da der Chip die Daten im Little-Endian-Format überträgt, konvertiert die Software die rohen Bytes in einen Integer und dividiert den Wert anschließend durch $256,0$. Dieses Verfahren berechnet den exakten prozentualen Ladezustand:

```
def get_soc(self):
    if not self.fuel_gauge_i2c: return None
    try:
        # Auslesen von 2 Bytes aus Register 0x06 (RepSOC)
        data = self.fuel_gauge_i2c.readfrom_mem(0x36, 0x06, 2)

        # Konvertierung von Little-Endian Bytes zu Integer
```

```
        soc_raw = int.from_bytes(data, 'little')

        # Umrechnung in Prozent gemaess Datenblatt
        soc = soc_raw / 256.0
        return soc
    except:
        return None
```

Listing 4.8: Register-Level I2C-Auslesung des MAX17262 Fuel-Gauge-ICs

Kinetische Interaktionssteuerung (Tap-Detection)

Eine knopflose Bedienung aktiviert die Spracherkennung. Das System liest hierfür den Beschleunigungssensor (LSM6DSOX) via SPI aus. Anstatt absolute Beschleunigungswerte zu nutzen, berechnet die Firmware den Ruck (Δg) zwischen zwei Frame-Messungen. Dieser Filter ignoriert normale Kopfbewegungen:

```
x, y, z = self.lsm.accel()
dx = abs(x - self.last_accel[0])
dy = abs(y - self.last_accel[1])
dz = abs(z - self.last_accel[2])
self.last_accel = (x, y, z)

delta_g = dx + dy + dz

if delta_g > 1.75:
    now = time.ticks_ms()
    diff = time.ticks_diff(now, self.last_tap_time)

    if 100 < diff < 600:
        self.send_tcp_packet(b'{"action": "tap"}\n')
        self.req_audio = True
        self.last_tap_time = 0
    elif diff > 600:
        self.last_tap_time = now
```

Listing 4.9: Berechnung von Delta-G zur Erkennung eines Double-Taps

Speichereffizientes Audio-Streaming

Eine Audioaufnahme (2,5 s, 16 kHz, Mono) erzeugt Datenmengen, die den freien Arbeitsspeicher des Nicla Vision übersteigen, falls das System sie am Stück puffert. Die Firmware implementiert daher einen *Circular Buffer* (`g_audio_chunks`).

Ein Hardware-Callback (`_audio_callback`) schreibt die Audiodaten während der Aufnahme iterativ in vorab allozierte 1024-Byte-Chunks. Die Nutzung eines `memoryview` verhindert, dass der Interpreter bei der Array-Zuweisung neuen Speicher alloziert (Zero-Copy-Prinzip). Nach der Aufnahme generiert die Firmware dynamisch einen 44 Byte großen RIFF/WAVE-Header mittels `struct.pack` und streamt diesen direkt über den TCP-Socket:

```
def _audio_callback(self, buf):
    global g_audio_written
    if not self.recording: return

    mv = memoryview(buf)
    l = len(mv)
    off = 0

    while l > 0:
        if self.rec_idx >= len(g_audio_chunks):
            self.recording = False
            break

        chunk = g_audio_chunks[self.rec_idx]
        space = AUDIO_CHUNK_SIZE - self.rec_offset
        amt = min(l, space)

        # Speicherzuweisung ohne RAM-Neuallokation (Zero-Copy)
        chunk[self.rec_offset : self.rec_offset+amt] = mv[off : off+amt]

        self.rec_offset += amt
        off += amt
        l -= amt
        g_audio_written += amt

        # Puffer-Index inkrementieren, wenn Chunk voll ist
        if self.rec_offset >= AUDIO_CHUNK_SIZE:
            self.rec_idx += 1
            self.rec_offset = 0
```

Listing 4.10: Speichereffizientes Befüllen des Circular Buffers mittels memoryview in MicroPython

Dynamisches Asset-Management des Akustikmodells

Das File-Management stellt ein spezifisches technisches Hindernis bei der Nutzung der lokalen Vosk-Spracherkennung unter Android dar. Die zugrundeliegende Vosk-Engine erfordert absolute Dateipfade auf einem unkomprimierten Dateisystem. Die Applikation löst dieses Problem durch einen asynchronen Entpackungsprozess beim Erststart der App. In einer IO-Coroutine iteriert die Funktion `copyAssets` rekursiv durch das komprimierte Modell-Verzeichnis der APK und kopiert die binären Dateien in den internen `filesDir`-Speicherbereich des Smartphones:

```kotlin
private fun copyAssets(path: String, out: String) {
    val list = assets.list(path) ?: return
    if (list.isNotEmpty()) {
        File(out).mkdirs()
        list.forEach { copyAssets("$path/$it", "$out/$it") }
    } else {
        val dir = File(out).parentFile
        if (dir != null && !dir.exists()) dir.mkdirs()
        assets.open(path).use { input ->
            FileOutputStream(out).use { input.copyTo(it) }
        }
    }
}
```

Listing 4.11: Rekursives, asynchrones Entpacken des Vosk-Modells in Kotlin

Audio-Verarbeitung und kognitive Entlastung

Auf dem Smartphone übergibt das System die resultierende WAV-Datei an die *Vosk Offline Engine*. Die Applikation liest die Datei schrittweise in 4096-Byte-Blöcken ein, um UI-Freezes zu verhindern. Nach der Auswertung generiert das System Warnungen über die `TextToSpeech`-API. Die App implementiert eine dynamische Cooldown-Logik, um den blinden Nutzer vor einer auditorischen Überlastung zu bewahren: Objekte in unkritischer Entfernung (über 50 cm) lösen eine Blockade von sechs Sekunden aus. Nähert sich ein Objekt jedoch in den kritischen Nahbereich (unter 50 cm), reduziert der Algorithmus den Cooldown drastisch und generiert eine direkte „Kollisionswarnung!“.

I/O-Auslagerung beim Bild-Streaming

Die Brille bietet zur Verifikation des Sichtfeldes die Funktion, ein Live-Bild an die Smartphone-App zu senden. Das direkte Versenden eines unkomprimierten Bitmaps über TCP führt jedoch zu einer sofortigen OOM-Exception.

Als Lösungsansatz nutzt die Firmware das Dateisystem als *Swap-Speicher*. Das Skript schreibt das Kamerabild mit stark reduzierter Qualität (40 %) als JPEG in den lokalen Flash-Speicher (`temp.jpg`). Danach zerstört die Firmware die Bildinstanz im RAM manuell (`del img`) und ruft den Garbage Collector auf. Erst im Anschluss liest das System die komprimierte JPEG-Datei iterativ in kleinen 1024-Byte-Blöcken aus und streamt diese über das Netzwerk:

```
def process_image(self):
    path = "temp.jpg"
    try:
        img = sensor.snapshot()
        img.save(path, quality=40) # Speichern im Flash

        # Manuelle Zerstörung der Bildinstanz zur sofortigen RAM-
            Freigabe
        del img
        gc.collect()

        size = os.stat(path)[6]
        self.send_tcp_packet(f"IMG_START:{size}\n".encode())

        # Iteratives Auslesen und Senden der komprimierten Datei
        with open(path, 'rb') as f:
            while True:
                chunk = f.read(1024)
                if not chunk: break
                self.send_tcp_packet(chunk)

        uos.remove(path) # Temporäre Datei sicher löschen
    except:
        pass
    finally:
        gc.collect()
```

Listing 4.12: Verwendung des Flash-Speichers als Swap zur Vermeidung von OOM-Exceptions

Benutzeroberfläche und App-Architektur (GUI)

Die Architektur der Applikation folgt dem *Single-Activity-Pattern*. Der Code unterteilt die eigentlichen Bildschirminhalte in modulare `Fragments`. Das `InfoFragment` visualisiert den aktuellen Verbindungsstatus sowie den via I2C ausgelesenen Akkustand der Hardware. Das `ImageFragment` empfängt Bilddatenpakete, dekodiert diese und rendert sie als Android `Bitmap`. Zudem implementiert dieses Fragment eine Funktion zur dauerhaften Speicherung von Umgebungsaufnahmen über die `MediaStore`-API des Betriebssystems (Scoped Storage). Das `AudioFragment` bietet eine Live-Transkription der Vosk-Engine. Dies ermöglicht Entwicklern, die Genauigkeit des akustischen Modells bei verschiedenen Umgebungsgeräuschen zu evaluieren.

4.2 Herausforderungen in der Entwicklung

Die Realisierung eines kompakten, tragbaren Assistenzsystems, welches rechenintensive Machine-Learning-Prozesse mit kontinuierlicher Netzwerkkommunikation vereint, bringt diverse interdisziplinäre Herausforderungen mit sich. Diese erfordern im Projektverlauf iterative Anpassungen an der Hardware-Architektur sowie tiefgreifende Software-Optimierungen.

4.2.1 Mechanische und geometrische Restriktionen

[Hier beschreibt Manuel seine Probleme beim 3D-Druck, Scharniere etc. sachlich.]

4.2.2 Energiemanagement im begrenzten Bauraum

[Hier beschreibt Fabian den Kompromiss mit den kleineren Akkus sachlich.]

4.2.3 Ressourcenlimits und Speicher-Fragmentierung (Software)

Auf softwareseitiger Ebene stellt der extrem begrenzte Arbeitsspeicher (RAM) des Nicla Vision die größte Hürde dar. Frühe Prototypen der Firmware stürzen regelmäßig mit *Out-Of-Memory* (OOM) Exceptions ab. Dies hat zwei Hauptursachen:

Erstens übersteigt die Pufferung von Audiodaten den verfügbaren Speicher. Das System verzichtet daher auf Standard-Audio-Bibliotheken zugunsten eines selbst entwick-

elten *Circular Buffers*. Dieser Buffer fragmentiert die Audiodaten über `memoryview`-Zuweisungen speichereffizient in 1024-Byte-Blöcke.

Zweitens verursacht die *Heap-Fragmentierung* Abstürze während der Bildverarbeitung. MicroPython alloziert Speicher dynamisch. Das ständige Erzeugen und Verwerfen von Objekten (wie `image.snapshot()` bei 240x240 Pixeln) hinterlässt mikroskopische Lücken im Speicher. Benötigt das FOMO-Modell anschließend einen zusammenhängenden Speicherblock für den Tensor, schlägt die Allokation trotz ausreichendem Gesamtspeicher fehl. Die Firmware implementiert daher eine strikte, deterministische Garbage-Collection-Strategie. Der Code erzwingt den Befehl `gc.collect()` nach jedem Frame-Snapshot und in den `finally`-Blöcken der Netzwerk-Routinen, um den RAM proaktiv zu defragmentieren.

Auf Seiten der Android-Applikation resultieren die größten Herausforderungen aus den restriktiven Energiesparmaßnahmen moderner Android-Versionen (Doze-Mode). Da die Applikation primär im Hintergrund bei gesperrtem Smartphone-Display operiert, blockiert das Betriebssystem standardmäßig den Empfang von UDP-Broadcasts. Dies verhindert den „Zero-Config"-Verbindungsaufbau. Die Implementierung eines `Foreground Services` in Kombination mit einem expliziten `WifiManager.MulticastLock` zwingt die Hardware jedoch dazu, eintreffende Netzwerkpakete zuverlässig an die Applikation durchzureichen. Dieser Mechanismus stellt die Systemstabilität letztlich sicher.

4.3 Tests und Validierung

Eine Reihe von technischen Validierungen belegt die Praxistauglichkeit des „Ascenta"-Prototyps. Die Tests fokussieren sich auf die Kernanforderungen eines Assistenzsystems: Latenzzeit, Zuverlässigkeit der Sensorik und energetische Ausdauer.

4.3.1 Netzwerklatenz und Streaming-Performance

Die Messung der Systemlatenz ermittelt die Zeitdifferenz zwischen der physischen Platzierung eines Objekts im Sichtfeld der Kamera und der Generierung des ersten Audio-Warnsignals (TTS) auf dem Smartphone. Dies stellt ein kritisches Sicherheitsmerkmal für blinde Nutzer dar.

Unter optimalen Laborbedingungen (kurze Distanz zwischen Brille und Smartphone, wenig Interferenz im 2,4 GHz WiFi-Band) ergibt sich eine durchschnittliche Latenzzeit

von etwa [XX] Millisekunden für die Bilderkennung und den reinen Telemetrie-Datentransfer (JSON). Der Transfer eines $2,5$-sekündigen Audio-Streams (16 kHz, Mono) via TCP an die Vosk-Engine inklusive anschließender Offline-Transkription dauert im Durchschnitt [XX] Sekunden. Diese Ergebnisse belegen, dass die gewählte „Split-Brain"-Architektur den Anforderungen an ein Echtzeitsystem gerecht wird.

4.3.2 Auswertung der kinetischen Interaktion (Tap-Detection)

Eine Versuchsreihe mit [XX] bewussten Eingaben (Taps) und [XX] simulierten Störbewegungen (schnelles Gehen, Kopfschütteln) evaluiert die Zuverlässigkeit der Double-Tap-Erkennung über die IMU (LSM6DSOX).

- **True-Positive-Rate:** Bei [XX] von [XX] beabsichtigten Doppeltipps aktiviert sich die Sprachaufnahme korrekt (Erkennungsrate von [XX] %).
- **False-Positive-Rate:** Während des simulierten Gehens über eine Distanz von [XX] Metern treten [XX] Fehlerkennungen auf.

Der gewählte Schwellenwert für die Ableitung der Beschleunigung ($\Delta g > 1,75$) filtert normale Kopfbewegungen somit adäquat heraus und setzt dennoch keine unverhältnismäßig hohe Schlagkraft des Nutzers voraus.

4.3.3 Energetische Evaluierung (Akkulaufzeit)

Die kontinuierliche Auslesung des integrierten MAX17262 Fuel-Gauge-ICs ermittelt die Laufzeit des Gesamtsystems. Das Testverfahren prüft zwei Betriebsmodi bei der verbauten Gesamtkapazität von 1240 mAh (Parallelbetrieb von zwei 620 mAh LiPo-Zellen):

Standby-Modus (Kamera deaktiviert, nur UDP-Heartbeat aktiv):
In diesem Modus beschränkt sich die Leistungsaufnahme auf die Aufrechterhaltung der WiFi-Verbindung. Der gemessene Entladestrom liefert eine theoretische Maximallaufzeit von [XX] Stunden.

Aktiver Modus (Kontinuierliche FOMO-Erkennung und TCP-Stream):
Im aktiven Modus wertet das Machine-Learning-Modell kontinuierlich Snapshots aus und sendet Datenpakete an die Applikation. Unter dieser maximalen Auslastung sinkt die Laufzeit auf etwa [XX] Stunden und [XX] Minuten. Dieser praktikable Wert ermöglicht den Alltagseinsatz während Einkäufen oder Wegstrecken problemlos. Die

Tests zeigen zudem keine signifikante thermische Drosselung (Thermal Throttling) des Cortex-M7 Prozessors.

5 Ausblick (Future Work)

Der aktuelle Prototyp des Assistenzsystems „Ascenta“ belegt die Machbarkeit einer dezentralen, offline-fähigen Smart-Glasses-Architektur. Um das System jedoch von einem funktionalen Prototyp zu einem marktreifen Endkundenprodukt weiterzuentwickeln, sind in zukünftigen Iterationen verschiedene Erweiterungen und Optimierungen in allen Fachbereichen erforderlich.

5.1 Software und Benutzeroberfläche

Die zukünftige Software-Entwicklung muss sich primär auf die vollständige Barrierefreiheit und erweiterte Navigationskonzepte konzentrieren.

5.1.1 Native Zero-UI-Applikation für blinde Nutzer

Die aktuelle Android-Applikation verfügt über eine grafische Benutzeroberfläche (GUI), die essenziell für das Debugging, die Evaluierung und die Ersteinrichtung durch sehende Begleitpersonen ist. In einer finalen Version für den Endanwender muss diese App jedoch als reine Hintergrundanwendung (*Zero-UI*) konzipiert werden.

Das visuelle Interface wird dabei vollständig obsolet. Die Applikation sollte sich nahtlos in native Accessibility-Dienste (wie Android TalkBack) integrieren und den Setup-Prozess (z. B. die Eingabe des WLAN-Passworts) über auditive Menüführung oder NFC-Tags abwickeln. Der Fokus liegt dabei auf maximaler Stabilität des Foreground-Services und einer Reduktion des Batterie- und Speicherverbrauchs des Smartphones, da keine visuellen Elemente gerendert werden müssen.

5.1.2 Kontinuierliches auditives Raum-Feedback (Leitstrahl-Prinzip)

Eine vielversprechende funktionale Erweiterung stellt die Implementierung eines dynamischen, akustischen Suchmodus dar. Sucht der Nutzer nach einem bestimmten Objekt (z. B. einem Stuhl oder einer Tür), könnte das System ein kontinuierliches Au-

diosignal generieren, das an die Funktion eines Geigerzählers oder eines Einparkhilfesensors angelehnt ist.

Wenn der Nutzer den Kopf schwenkt und das gesuchte Objekt in das Sichtfeld der Kamera gerät, passt der Algorithmus die Frequenz (Tonhöhe) und das Intervall (Pings pro Sekunde) des Audiosignals in Echtzeit an. Je zentrierter das Objekt im Sichtfeld ist und je geringer die Distanz (gemessen durch den Time-of-Flight-Sensor) wird, desto intensiver wird das akustische Feedback. Dies würde blinden Nutzern eine hochpräzise, intuitive Zielführung ermöglichen, ohne sie mit komplexen, gesprochenen Sätzen zu überlasten.

5.2 Künstliche Intelligenz und Modell-Training

5.2.1 Erweiterte Objekterkennung für Alltagsgegenstände

Derzeit ist das System auf die Erkennung von Personen optimiert. Für eine umfassende Alltagshilfe muss das zugrundeliegende Dataset massiv erweitert werden. Die Firmware und die App-Logik sind bereits so abstrahiert, dass sie beliebige Labels verarbeiten können. Zukünftige Iterationen müssen daher trainiert werden, um eine Vielzahl von Hindernissen und Zielen (z. B. Treppenstufen, Ampeln, Zebrastreifen, Türen, Sitzgelegenheiten oder herannahende Fahrzeuge) zuverlässig zu klassifizieren.

5.2.2 Weiterentwicklung des Machine-Learning-Modells (Edge Impulse)

[Hier fügt Fabian seinen Ausblick zum Modell-Training ein. Mögliche Themen: Nutzung von leistungsstärkeren Modellen als FOMO, Transfer-Learning mit größeren Datensätzen, Hardware-Beschleunigung durch Ethos-U55 NPU (falls anwendbar), oder das Sammeln von spezifischen Blinden-Datensätzen.]

5.3 Hardware und Elektronik

5.3.1 Optimierung der Schaltungstechnik und Sensorik

[Hier fügt Fabian seinen Ausblick zur Elektronik ein. Mögliche Themen: Integration eines dedizierten Audio-Chips für besseres Mikrofon-Handling, Umstieg auf ein Custom-PCB (Printed Circuit Board) anstelle von Entwickler-Boards, Optimierung des Ladereglers (QuickCharge) oder Einsatz von Solid-State-Batterien für mehr Sicherheit.]

5.4 Mechanik und Gehäusedesign

5.4.1 Weiterentwicklung des 3D-Modells und der Ergonomie

[Hier fügt Manuel seinen Ausblick zum Gehäusedesign ein. Mögliche Themen: Nutzung von flexibleren Materialien (TPU/Resin-Druck), Gewichtsreduktion durch dünnere Wandstärken, IP67-Zertifizierung (Wasserdichtigkeit) durch bessere Dichtungen oder ein modulareres Klick-System für austauschbare Akkus.]

6 Verwandte Arbeiten (Related Work)

Die Entwicklung von Assistenzsystemen für sehbeeinträchtigte Personen ist ein etabliertes Forschungs- und Entwicklungsfeld. Um die Positionierung und den Mehrwert des „Ascenta“-Projekts zu verdeutlichen, werden in diesem Kapitel bestehende Lösungsansätze analysiert und dem hier entwickelten System gegenübergestellt. Die existierenden Systeme lassen sich grob in klassische Hilfsmittel, softwarebasierte Smartphone-Lösungen und dedizierte, kommerzielle Wearables unterteilen.

6.1 Klassische Mobilitätshilfen

Das mit Abstand am weitesten verbreitete Hilfsmittel ist der klassische weiße Langstock. Er ermöglicht durch taktiles Abtasten die Erkennung von Bodenhindernissen, Treppenstufen und Bordsteinkanten. **Abgrenzung:** Der Langstock ist eine essenzielle Basisversorgung, weist jedoch einen entscheidenden blinden Fleck auf: Hindernisse auf Rumpf- und Kopfhöhe (z. B. offene Fenster, Verkehrsschilder oder Äste) werden nicht erfasst, was ein erhebliches Verletzungsrisiko birgt. „Ascenta“ versteht sich nicht als Ersatz für den Langstock, sondern als komplementäres System, welches genau diesen toten Winkel durch den Einsatz des Time-of-Flight-Sensors (ToF) abdeckt.

6.2 Smartphone-basierte Applikationen

In den letzten Jahren haben sich Smartphone-Apps etabliert, die die Handykamera zur Umgebungsanalyse nutzen. Prominente Beispiele sind *Seeing AI* (Microsoft) oder *Be My Eyes*. Diese Applikationen können Texte vorlesen, Währungen erkennen oder Videotelefonate mit sehenden Freiwilligen aufbauen. **Abgrenzung:** Der größte Nachteil dieser Lösungen ist mangelnde Ergonomie. Der Nutzer muss das Smartphone permanent in der Hand halten und aktiv auf Objekte richten, was die Hände für die Nutzung des Langstocks oder von Blindenführhunden blockiert. Zudem basieren diese Apps fast ausschließlich auf Cloud-Computing. Dies erfordert eine konstante Breitband-Internetverbindung und wirft datenschutzrechtliche Bedenken auf,

da sensible Videoströme der Umgebung an externe Server gesendet werden. „Ascenta“ löst dieses Problem durch den „Hands-free“-Wearable-Ansatz und die strikte, lokale Datenverarbeitung (Offline-Edge-AI).

6.3 Kommerzielle Smart Glasses und Wearables

Auf dem kommerziellen Markt existieren hochpreisige Speziallösungen. Das bekannteste System ist *OrCam MyEye*, ein kompaktes Kameramodul, das magnetisch an herkömmlichen Brillen befestigt wird. Es liest Texte vor und erkennt Gesichter. Ein weiteres Beispiel sind die *Envision Glasses*, welche auf der Google Glass Enterprise Edition 2 basieren und komplexe Szenenbeschreibungen mithilfe von Künstlicher Intelligenz liefern. **Abgrenzung:** Obwohl diese Systeme technologisch hochentwickelt sind, weisen sie massive Hürden in der Zugänglichkeit auf. Die Anschaffungskosten liegen typischerweise zwischen 2.500 und 4.000 Euro, was sie für einen Großteil der Betroffenen – insbesondere in Entwicklungsländern oder ohne vollständige Kostenübernahme durch Krankenkassen – unerschwinglich macht.

6.4 Positionierung des Ascenta-Projekts

Das Projekt „Ascenta“ positioniert sich als Brückenschlag zwischen kostengünstiger Machbarkeit und datenschutzkonformer Hochtechnologie. Durch die „Split-Brain“-Architektur wird die teure und wärmeintensive Rechenhardware (wie sie in den Envision Glasses verbaut ist) durch das ohnehin beim Nutzer vorhandene Smartphone ersetzt.

Im Gegensatz zu den kommerziellen Systemen belaufen sich die Materialkosten für den Ascenta-Prototyp (Arduino Nicla Vision, LiPo-Akkus, 3D-Druck-Filament) auf unter 150 Euro. Gleichzeitig garantiert die lokale Ausführung des FOMO-Modells sowie der Vosk-Spracherkennungs-Engine eine 100-prozentige Offline-Funktionalität, was die Reaktionszeiten (Latenz) minimiert und die Privatsphäre des Nutzers und dessen sozialen Umfelds absolut schützt.

7 Conclusion

8 Project Management

8.1 Planning

Die organisatorische Planung und Umsetzung des Projekts „Ascenta“ basierte auf einer meilensteinorientierten, agilen Vorgehensweise. Zu Beginn des Projekts (Vorbereitungsphase) lag der Fokus auf der grundlegenden organisatorischen Planung, der Definition von Grob- und Feinzielen sowie der Recherche und Beschaffung der elektronischen Komponenten.

Die formelle Strukturierung der Arbeitsschritte und die Einreichung der Projektmeilensteine erfolgten über das schulspezifische ABA-Portal. Um eine strukturierte Arbeitsteilung zwischen den Bereichen Software, Elektronik und 3D-Druck zu gewährleisten, wurde der Projektplan in wöchentliche Sprints (Kalenderwochen, KW) unterteilt. Zur Überwachung des Fortschritts und der individuellen Arbeitslast wurde eine detaillierte Zeiterfassung (Timesheet) mittels Tabellenkalkulation implementiert, welche die geplanten Freizeitstunden dem tatsächlichen Aufwand (Soll/Ist-Vergleich) kontinuierlich gegenüberstellt.

8.2 Evaluation

Die wöchentliche Evaluierung der Tätigkeitsberichte ermöglichte eine präzise Überwachung des Projektfortschritts. Ein Abgleich der geplanten Ressourcen mit dem tatsächlichen Aufwand zeigt insbesondere in der initialen Projektphase eine deutliche Diskrepanz: Während für die Projektdefinition anfänglich 6 Stunden pro Teammitglied veranschlagt wurden, belief sich der tatsächliche Aufwand auf 20 Stunden pro Person. Diese anfängliche Unterschätzung ist typisch für komplexe Hardware-Software-Kombinationen, da grundlegende Architektur-Entscheidungen zeitintensiv sind.

In den darauffolgenden Wochen (beispielsweise in KW 37 und 38) stabilisierte sich die Arbeitslast und entsprach mit durchschnittlich 6 Stunden pro Teammitglied exakt den Planwerten. Die konsequente Dokumentation der wöchentlichen Ist-Zustände – wie etwa der erfolgreiche Test des Spannungsbetriebsbereichs der Akkus oder die Evaluation der Micro-Speech-Software – half maßgeblich dabei, technische Hürden frühzeitig

zu identifizieren und den Projektplan dynamisch anzupassen.

8.3 Timesheet

Die nachfolgenden Unterschriften bestätigen die Korrektheit der in der internen Zeiterfassung dokumentierten, individuell erbrachten Arbeitsstunden für die Diplomarbeit.

8.3.1 Liam Vendel

Braunau am Inn, March 11, 2026 Liam Vendel
Ort, Datum

Gesamtarbeitsaufwand: [XX] Stunden

8.3.2 Manuel Zauner

Gesamtarbeitsaufwand: [XX] Stunden

Braunau am Inn, March 11, 2026 Manuel Zauner _____
Ort, Datum *Unterschrift*

8.3.3 Fabian Lindlbauer

Gesamtarbeitsaufwand: [XX] Stunden

Braunau am Inn, March 11, 2026	Fabian Lindlbauer	_____
Ort, Datum		*Unterschrift*

Acknowledgements

List of Figures

Quellcodeverzeichnis

3.1 Auszug der initialen GATT-Service-Definitionen für die alte BLE-Architektur 8
3.2 RAM-Allokation und Pufferung der Audiodaten in der initialen Firmware 9
3.3 Die BLE-Sendeschleife mit künstlicher Drosselung zur Puffer-Schonung 10

4.1 Akquirierung des MulticastLocks und Start des Foreground Services . . 13
4.2 Ausschnitt der asynchronen GATT Callback-Struktur in Kotlin 13
4.3 Asynchroner UDP-Discovery-Listener in Kotlin 14
4.4 Dynamisches Stream-Multiplexing zur Trennung von Text- und Binärdaten 15
4.5 Heartbeat-Coroutine zur Überwachung der Verbindungsstabilität 16
4.6 Sicherer Kontextwechsel zwischen IO-Thread und Main-Thread in Kotlin 16
4.7 Extraktion der Kameradaten und asynchroner JSON-Telemetrie-Stream 17
4.8 Register-Level I2C-Auslesung des MAX17262 Fuel-Gauge-ICs 18
4.9 Berechnung von Delta-G zur Erkennung eines Double-Taps 19
4.10 Speichereffizientes Befüllen des Circular Buffers mittels memoryview in MicroPython 20
4.11 Rekursives, asynchrones Entpacken des Vosk-Modells in Kotlin 21
4.12 Verwendung des Flash-Speichers als Swap zur Vermeidung von OOM-Exceptions 22

LIAM VENDEL

Gampern, Österreich | +43 664 1851637 |
Liamvendel06@gmail.com
github.com/schlaimwendla

PROFIL

Angehender Elektronik-Absolvent der HTBLA Braunau mit dem Schwerpunkt **Coding & AI**. Spezialisiert auf Backend Engineering und verteilte Systeme. Fundierte Kenntnisse in Java (OOP, Concurrency) und Low-Level Architektur. Begeisterter Linux-Nutzer (Arch/Garuda) mit Fokus auf Clean Code und Performance.

TECHNISCHE SKILLS

Programmierung:	Java (Fortgeschritten), Python, C++, MicroPython, Assembly (Memory Management).
Systeme & Tools:	Linux (CLI Experte), Docker, Git, Spring Boot, REST APIs, Bash Scripting.
KI & Hardware:	Nicla Vision, ESP32, Raspberry Pi, Edge Impulse, Wit.ai (NLP).
Sprachen:	Deutsch (Muttersprache), Englisch (Fließend).

DIPLOMARBEIT: "ASCENTA"

Smarte Assistenzbrille mit Sprachsteuerung *seit 2025*

- Entwicklung einer synchronisierten **UDP/BLE-Kommunikationsstruktur** zwischen Wearable (Nicla Vision) und Mobile App (Android/Kotlin).
- Integration von **Edge AI** zur Latenzoptimierung (Objekterkennung via Edge Impulse).
- Implementierung einer Offline-Sprachsteuerung zur Gewährleistung maximaler Privatsphäre.

PROJEKTE & HIGHLIGHTS

IoT Light Control System *Raspberry Pi / Python*

- Entwicklung eines REST-API Backends zur modularen und drahtlosen Steuerung von Lichtmodulen.

Automatic Guitar Tuner *Arduino / Signalverarbeitung*

- Design eines elektromechanischen Stimmgeräts auf Basis von Frequenzanalyse und Feedback-Loops.

European Space Camp 2025 *Andøya Space Center, Norwegen*

- Auswahl für das internationale Forschungs-Camp (Fokus: Sounding Rockets & Systems Engineering).

AUSBILDUNG

HTBLA Braunau **2021 – 2026**

Höhere Technische Bundeslehranstalt für Elektronik und Technische Informatik
Schwerpunkt: **Coding & AI** (Voraussichtlicher Abschluss: Juni 2026)

INTERESSEN & ENGAGEMENT

- **Open Source:** Linux-Kernel & System-Optimierung (Btrfs-Subvolumes, Kernel-Tuning).
- **Sport:** Aktiver Volleyball-Spieler und Jugend-Coach im Verein.
- **Raumfahrt:** NASA Extracurricular Kurse in Weltraumphysik und orbitaler Mechanik.

Manuel Zauner

Höfter Straße 70a

5280 Braunau am Inn

Tel: 0677 62550900

E-Mail: manuel.zauner.2000@gmail.com

Lebenslauf

Angaben zur Person

Name:	Manuel Zauner
Geburtsdatum:	19.01.2007
Geburtsort:	Braunau am Inn
Staatsbürgerschaft:	Österreich

Familiensituation:

Vater:	Christian Zauner (Technischer Angestellter, Fa. Spiratec)
Mutter:	Bettina Zauner (Personalverrechnung, Fa. EKB)
Geschwister:	Thomas Zauner (Gymnasium Braunau)

Schulbildung

Art/Dauer der Schule:	2013 – 2017	VS Braunau-Laab
	2017 – 2021	Gymnasium Braunau
	2021 – laufend	HTL Braunau (Coding & AI)
Lieblingsfach:	PBE_3 (Prototypenbau elektronischer Systeme)	

Berufserfahrung

Ferialjob:	August 2023	4 Wochen in der Firma Dräxlmaier
	August 2024	4 Wochen in der Firma Dräxlmaier

Sonstige Angaben

Besondere Fähigkeiten:	EDV, gute Menschenkenntnisse
Hobbys:	Basketball, Krafttraining, Tennis, Computer

Lebenslauf

Name: Fabian Lindlbauer
Geburtsdaten: 24.07.2007
Anschrift: Moritzhub 8, 4950 Altheim
Telefon: 06504307493
E-Mail: fabian.lindlbauer007@gmail.com

Schulausbildung:

2013-2017	Volksschule Altheim
2017-2021	Mittelsschule Altheim
2021-	HTL Braunau

Berufserfahrung:

Juli-Juli 2023	Hargassner Weng	Ferialpraktikum
Juli-Juli 2024	SGS Dorf an der Pram	Ferialpraktikum
Juli-Juli 2025	HTL Braunau Schulpartnerschaft Nicaragua	

Fremdsprache:

Englisch B2

ECDL
Führeschein B

Altheim, 11.09.2025

Fabian Lindllbauer